

CS7641 Supervised Learning Project

Abdelmalak, Abanoub
Georgia Institute of Technology
abanoub_abdelmalak@gatech.edu

Abstract

This study analyzes the performance of various supervised machine-learning algorithms on different datasets. We focus our evaluation on the following models: Neural Network, K-nearest neighbor, and Support Vector Machine. Two distinct datasets are considered: the tennis player winners dataset and the stock predictions dataset. We evaluate the effectiveness of these techniques using metrics like accuracy, precision, recall, F1 score, and computation time, taking into account factors like dataset size, feature selection, and preprocessing. The results give insights into selecting appropriate machine-learning strategies based on dataset characteristics, thereby improving the efficiency and effectiveness of supervised learning applications. This research adds to our understanding of algorithmic performance across different data contexts, offering guidance for future machine-learning practices and studies.

Index Terms—Supervised Learning, F1 Score, Stocks Prediction, Tennis Match Prediction.

I. Introduction

Machine learning is a rapidly evolving field and there are several supervised learning models available that are widely used for their predictive capabilities in various applications. This paper presents a comprehensive performance analysis of different models using two distinct datasets. The first dataset is the tennis match-winning prediction dataset and the second dataset is a stock dataset. This choice of datasets provides an opportunity to explore the adaptability and efficiency of the algorithms in learning for both. The tennis matches dataset is ideal for assessing the models' performance in a controlled environment with a clear structure and low noise, while the stocks dataset presents a challenge due to its inherent noise and non-stationarity, testing the models' ability to decipher more subtle patterns.

Georgia Institute of Technology, CS7641 ML TA and Instructors team.

The paper first introduces the datasets used in this study, detailing their characteristics and relevance. It then discusses the selected algorithms for analysis, outlining their theoretical foundations and justifying their inclusion. Finally, it presents the performance analysis results, including learning curves and other relevant metrics, as well as a comparison of the models' performance against the two selected datasets.

II. Datasets Introduction

After exploring and experimenting with different datasets, we eventually selected the tennis matches dataset and the stock market prediction dataset. This section introduces the dataset sources, and the corresponding features preprocessing, and then, finally, drives a comparison of the datasets from the machine learning usability perspective.

A. Stock Market Dataset

1) **Overview:** Stock market prediction (SP) is a fascinating field in financial data analysis and has been a subject of machine-learning algorithms study for a long time. This work has addressed this problem by analyzing data from 2009 to 2021 for stock symbols "AAPL" and "JPM."

2) **Data Source:** The source for the database is [here](#).

3) **Dataset Features Details:** The stock market relies on several technical indicators that signal the buy and sell signals. These indicators are used as features for a stock on a given day, and the corresponding target value is whether to buy or sell. Key indicators include:

- **Moving Averages (MA):** Shows the average price of a stock over a certain period.
- **Bollinger Bands:** Measures volatility.
- **Relative Strength Index (RSI):** Helps determine whether a stock is overbought or oversold.
- **Volume Weighted Average Price (VWAP):** Considers both the price and volume of trades, providing insight into the overall trend of a stock.
- **Other Common Indicators:** Include MACD (Moving Average Convergence Divergence), Stochastic Oscillator, and Fibonacci retracement levels.

The following features are used to calculate the buy/sell signal lines:

- Simple Moving Averages (SMA) with window referring to number of averaging days: SMA10, SMA20, and SMA50.
- Exponentially Weighted Moving Averages (EWMA) with window referring to number of averaging days: EWMA12, and EWMA26.
- Returns and Indicators: 26_day_return and 12_day_return, MACD, mad_histogram, and signal_line.

The generated buy/sell signal results are represented as:

- **1** : Long the stock
- **-1**: Short the stock
- **0** : Hold the position

In our analysis, we focus on how well the predictions of the machine learning performance can better predict future signals based on previous history compared to the signal generated from previously mentioned features.

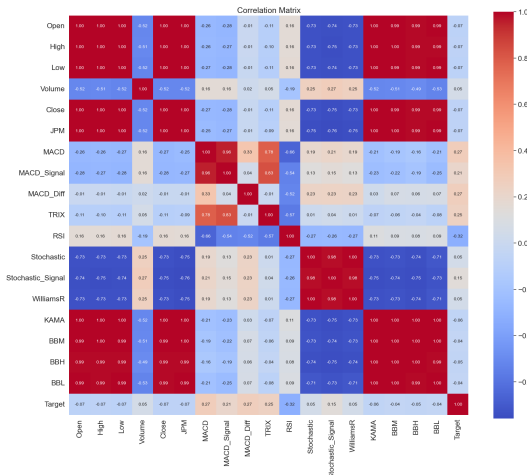


Fig. 1: SP-JPM Feature Heat Map

Figure 1 presents the correlation matrix between the features and the target. This heat map is a good indication of noise in the dataset. When analyzing the bottom row "Target" to the features columns, we see the feature "MACD" has the highest score tied with "MACD_Diff" in correlation with the "Target" column. This makes sense since some of these features are driven from each other and the final "Target" is heavily calculated from them as well while prioritizing the "MACD" signal for changing the stock position (i.e. "Target"). Looking at how the "MACD" feature is affected by other features, we can fire the chain reaction to see the correlation of the "MACD" signal with the "Trix" value and so on comparing the highest correlation values in columns and features. As we mentioned, these features are all driven from the "AdjClose" (i.e. "JPM" feature), therefore, the correlation between the "JPM" and all the other features is almost close to "1".

B.Tennis Matches Dataset

1) **Overview:** Tennis men's sport (TLS) is one of the most popular sports in the world, and its popularity has aroused great interest in the betting market in recent years. However, in tennis, unlike other popular sports such as soccer, analytics is still in a very early stage. This fact presents an excellent opportunity to make a difference in the betting market, especially since the sports betting market size is experiencing tremendous growth of \$167.66 billion by 2029, estimating a CAGR (Compound Annual

Growth Rate) of 10.26% as per one of the latest market researches on [Data Bridge](#).

2) **Data Source:** The database introduced here is a pre-processed version of the tennis match database collected in this [repository](#) repository by "JeffSackman." For installation details, please follow the readme file commands attached in the project repository [here](#).

In the final delivered code, all the needed data files have been extracted already, and the evaluation file can be invoked separately without having to install the original database.

3) **Dataset Features:** Features list available for this database that are considered important are:

- Match-based Features (match format, surface, indoor/outdoor)
- Unique Player Features (height, age, One backhand or two)
- Current Ranking (official and ELO ranking)
- Player Win % (General win %, win % under the same match format, in the same tournament surface, and in this format)

Combining these features provides great depth of predictable and hyper-parameter tuning for the algorithms (more details will be discussed later).

The generated result in this case is: given two players in a row,

- 1 means player1 has won,
- 2 means player2 has won,
- And 0.5 means there was a draw.

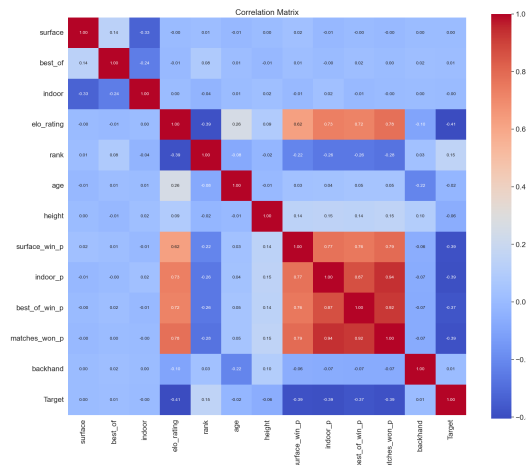


Fig. 2: TLS Feature Heat Map

Figure 2 represents the features' correlations for TLS. Unlike SP dataset, most of the features here are unrelated to one another. One exception can be found for the relationship between "surface_win_p" (the percentage of winning according to the surface) and whether or not the match is indoor or outdoor as well as the overall "matches_won_p". This means that each player has a signature of winning in a specific environment over the others which can be very powerful when analyzing the models' performance in the later sections.

C.Comparison Analysis

1) **Purpose: ML Prospective:** From a machine learning perspective, the Tennis Match and Stock Market datasets highlight the adaptability of algorithms to varied challenges. The Tennis dataset, with its clear-cut classification tasks, offers a less

noisy environment ideal for assessing model accuracy. In contrast, the Stock Market dataset, rife with noise and volatility, presents a complex regression scenario that tests the predictive abilities and noise management of models. This comparison underscores the need for models to handle both categorical classification and the intricacies of time-series regression, reflecting real-world predictive analytics applications.

2) **Why do they have different results?:** This is depending on the feature types and their corresponding ranges. In the SP dataset, most of the features are numerical continuous features derived from the "Adjusted Close" value of the stock for a given day. In the TLS dataset, on the other hand, the features are categorical and data are not related to each other directly. The below points highlight the general view of this comparison.

- **Nature of Prediction:** Tennis data typically leads to classification problems (predicting a winner or loser), which may have a clear decision boundary based on player statistics. Stock data involves regression problems (predicting continuous financial figures), where the relationship between features and targets can be more complex and less distinct.
- **Data Patterns and Consistency:** Tennis match outcomes are influenced by fewer variables and can display more consistent patterns, such as a top-ranked player generally winning against lower-ranked players. Stock market predictions are affected by a multitude of unpredictable factors like economic indicators, global events, and market sentiment, making patterns more complex and often non-linear.
- **Volatility:** Tennis data volatility is generally lower and contained within the scope of player performance and match conditions. Stocks are inherently volatile, and their predictions are subject to rapid and unexpected changes, introducing higher levels of uncertainty for ML models.
- **Feature Correlation:** In tennis data, features like player rankings and historical match results may have a direct and interpretable correlation with outcomes. Stock data may contain features with indirect, hidden, or non-stationary correlations, posing challenges in feature selection and engineering for ML models.
- **Noise and Outliers:** Tennis datasets may have fewer outliers or noise, as the data points (matches) are relatively controlled events. Stock datasets can be noisy with many outliers due to the erratic nature of financial markets and external influences.
- **Time-Series Analysis:** Tennis match predictions do not usually require complex time-series analysis as each match is an independent event. Stock market data is time-dependent, and the success of ML models heavily relies on their ability to capture temporal dynamics.
- **Overfitting Risks:** The risk of overfitting may differ; in tennis data, a model might overfit to specific players' data without generalizing well to others. In stock market data, models may overfit to past market trends that do not necessarily predict future movements, requiring careful regularization and validation techniques.
- **Data Availability and Granularity:** Tennis datasets are typically event-based with data available at the match level, which can lead to a more granular analysis for each event. Stock market datasets are often available at different time intervals (minute, hourly, daily), which can affect the granularity of predictions and the model's ability to react to market changes.

3) **Data Size:** Data set sizes are summarized in table I presented below. The dataset is split by 20% for testing and 80% for training.

Dataset	Training Set	Testing Set	# Features	Overall
SP-JPM	2159	926	18	3085
TLS	11234	4815	12	16049

TABLE I: Data Size Summary

III. Experiments Approach

A. Overview

This git repository has all the experiments compiled. In it, we define a base abstract model that other algorithms inherit and develop on top of it. This section introduces the algorithms examined, the corresponding hyper-parameters tuned and summarizes the final tuning ranges of these parameters. A lot of the definitions here are cited from [sklearn documentation](#).

1) **Decision Tree (DT):** While the decision Tree algorithm is a very simple learner, it's also very effective in studying the data and the feature importance. Hyper-parameters tuned are:

- Max depth: The maximum depth of the tree.
- Criterion: The function to measure the quality of a split.
- Min Samples Leaf: The minimum number of samples required to be at a leaf node.

2) **Support Vector Machine (SVM):** SVM classifies the data by finding an optimal line or hyperplane that maximizes the distance between each class in N-dimensional space (refer to [here](#)). Hyper-parameters tuned are:

- C: Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive.
- Kernel: Specifies the kernel type to be used in the algorithm.

3) **k-Nearest Neighbor (kNN):** kNN is a non-parametric supervised learning algorithm that predicts the correct class for test data by calculating the distance between the test data and the training points, selecting the K closest points, and assigning the label most frequent among those neighbors (refer to [here](#)). Hyper-parameters tuned are:

- N Neighbors: Number of neighbors to use by default for neighbors queries.
- Algorithm: used to compute the nearest neighbors.
- Metric: specifies the distance metric used.

4) **Neural Networks (NN):** A neural network is a group of interconnected units called neurons that send signals to one another, mimicking the operations of an animal brain to recognize relationships between vast amounts of data (refer to [here](#)). Hyper-parameters tuned are:

- Alpha: Strength of the L2 regularization term. When added to the loss, the L2 regularization term is divided by the sample size.
- Learning Rate: Learning rate schedule for weight updates.
- Hidden Layer Sizes: The i-th element represents the number of neurons in the i-th hidden layer.
- Activation: Activation function for the hidden layer.

5) **Boosting (AdaBoost):** AdaBoost (short for Adaptive Boosting), is a statistical classification meta-algorithm that combines the outputs of other learning algorithms (weak learners) to improve performance, with subsequent learners adjusted in favor of instances misclassified by previous classifiers (refer to [here](#)). Hyper-parameters tuned are:

- **N Estimators:** The maximum number of estimators at which boosting is terminated.
- **Learning Rate:** Weight applied to each classifier at each boosting iteration. A higher learning rate increases the contribution of each classifier.

B.Summary

Table II presents summary for hyper-parameters tuned and their corresponding tuning ranges. Each parameter affects the performance of the learner differently as we will see in the Results section. Due to the size of the TLS dataset, some of the hyper-parameters space has been narrowed down (after several iterations) to the one presented in the table.

Algorithm	Hyper Parameters	
	Parameter Name	Ranges
DT	Max Depth	np.arange(1, 50, 1)
	Criterion	['gini', 'entropy']
	Min Samples Leaf	np.arange(1, 20, 2)
SVM	C	np.logspace(-1, 1, 3)
	Kernel	['rbf', 'poly']
kNN	N Neighbors	np.arange(1, 50, 1)
	Algorithm	['auto', 'ball_tree', 'kd_tree', 'brute']
	Metric	['euclidean', 'manhattan', 'chebyshev', 'minkowski']
NN	Alpha	np.arange(0.0001, 0.001, 0.0001)
	Learning Rate	['constant', 'invscaling', 'adaptive']
	Hidden Layer Sizes	[(5), (5, 5), (6, 6), (7, 7), (7, 7, 7), (8, 9), (9, 2), (10, 10, 10), (50, 50, 50)]
AdaBoost	Activation	['tanh', 'relu']
	N Estimators	np.arange(1, 50, 1)
	Learning Rate	[0.1, 0.5, 1, 1.5, 2]
	Base Estimators	[DT, kNN, NN, SVM] with best parameters.

TABLE II: Experiments Parameters Overview

Below general definitions are used:

- **Precision** quantifies the number of true positive predictions that belong to positive class. (Equation 1)

$$Precision = \frac{TP}{TP+FP} \quad (1)$$

- **Recall** (also known as sensitivity or true positive rate) measures the proportion of actual positives that were predicted correctly. It takes into account false negatives, which are cases that should have been flagged for inclusion but weren't. (Equation 2)

$$Recall = \frac{TP}{TP+FN} \quad (2)$$

- **Accuracy** is a simple metric for evaluating classification models. It calculates the ratio of the number of correct predictions to the total number of predictions:

$$Accuracy = \frac{TP+TN+FP+FN}{TP+TN} \quad (3)$$

- **F1 Score** balances both precision and recall, providing a comprehensive evaluation of a model's performance. (Equation 4)

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (4)$$

- **Learning Curve (LC)** demonstrates if the data used is enough or do we need to add further data to have a more conclusive analysis.

- **Validation Curve (VC)** explains the relationship between the bias/variance and the model performance with respect to the model hyper-parameter value.

Where:

- TP = True Positives, TN = True Negatives
- FP = False Positives, FN = False Negatives

IV. Results and Discussion

A.Features Analysis

To determine the best model performance, we selected important features for each model using the feature importance parameter from a decision tree model results (trained model to study the dataset along with domain knowledge). A box plot has been generated for analysis, comparing the chosen feature groups.

Features Set	DataSet	
	SP	TLS
FeatSet1	MACD, MACD_Signal, MACD_Diff, TRIX, RSI, Stochastic_Signal, Stochastic, WilliamsR, KAMA, BBM, BBH, BBL	surface, best_of, indoor, elo_rating, rank, age, height, surface_win_p, indoor_p, best_of_win_p, matches_won_p, backhand
FeatSet2	MACD, MACD_Signal, MACD_Diff, TRIX, RSI, Stochastic, Stochastic_Signal, WilliamsR	surface, best_of, indoor, elo_rating, rank, surface_win_p, indoor_p, best_of_win_p, matches_won_p
FeatSet3	KAMA, BBM, MACD_Signal, RSI, Stochastic_Signal	surface, best_of, indoor, elo_rating, rank, age, height

TABLE III: Experiments Features Sets Columns

Table III shows the feature set groups that are being used in each dataset. These columns are then used to compare performance and examine the average fluctuation when adding each feature group separately.

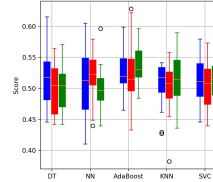


Fig. 3: SP FeatSets

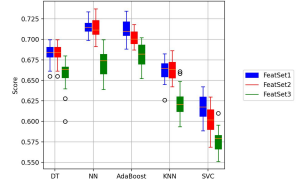


Fig. 4: TLS FeatSets

The plots 3 and 4 illustrate the relationship between the average and standard deviation of the accuracy scores across 10-fold cross-validation sets. For fig 3, the feature sets added do not have a lot of value in terms of performance, and box plots are nearly similar for all FeatSets. On the other hand, fig 4 shows a clear identification of added value when using the extra features.

This exercise analyzes the curse of dimensionality on the models' performance. The SP dataset suffers, to some degree, from the curse of dimensionality because different feature groups perform almost similarly, and therefore, a lower number of dimensions would be preferred. The use of domain knowledge can explain this. Although the signals created from the "AdjClose" price may not directly impact the stock position, these signals (like "BBM" and so forth) still contribute to the market simulation and, therefore, have a lower impact on the outcome. Relating to the dimensions correlation matrix, in 1, we can see that not all the features directly affect the final "Target" concept. Therefore adding it won't impact the final performance significantly. TLS, on the other hand, benefits from the dimensions added as they contribute to the player identity affecting how the model interprets the relationship between the player signature and match winning.

For the following analysis subsections, we applied grid search and found the best model using the tuning parameters mentioned in II.

B.DT: Analysis

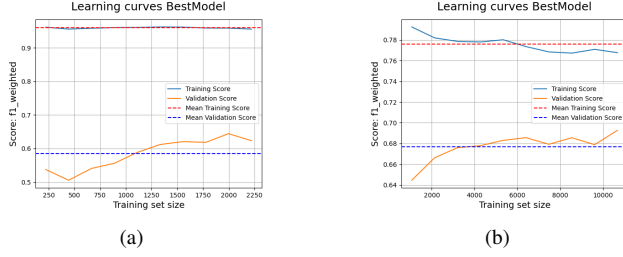


Fig. 5: DT LC (a) SP-JPM (b) TLS

1) **Learning Curves:** Figure 5(a) shows the effect of adding further data on the validation and training sets. In SP dataset, it's more likely that adding further data might impact the overall performance of the model. Since in SP dataset, we are dealing with time-series analysis, the data can not be shuffled and they are timely dependent on each other. Therefore, adding further data indicates more knowledge of a specific stock history and it also shows a strong belief in the stock's company against economic behaviors. Since we are dealing with a non-ideal world, we do not want to commit too much to the data and believe in the history very much or this might be misleading for future predictions.

In contrast, figure 5(b) shows a strong bond between the performance of the validation and training sets. Looking at the model's ability to predict even with only 4000 samples of the 14000 original dataset, we can deduce that the DT model does not need a lot of data to stabilize. The stability here can be seen as well in the way the fluctuation of the training and validation sets are getting closer to each other within a small range of change.

2) **Validation Curves:** For the DT model, we use the parameters set in table II with their ranges, and the following curves were generated.

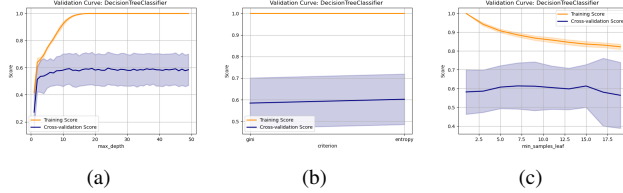


Fig. 6: SP-JPM/DT (a) Max Depth VC (b) Criterion VC (c) Min Samples Leaf VC

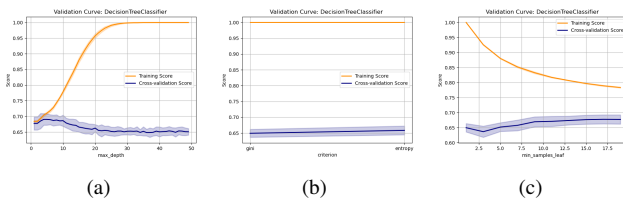


Fig. 7: TLS/DT (a) Max Depth VC (b) Criterion VC (c) Min Samples Leaf VC

Upon comparing figs 6(a) & 7(a), we can see an early convergence of the model concerning different "max_depth" for

the different datasets. In the SP dataset, the testing set curve flattens at around "20" but the validation set flattens earlier at the point "10" for fig 6(a). TLS curves in contrast converged much earlier at around "5" in 7(a). Max depth affects the model complexity, and since a simpler model is desirable to avoid overfitting, lower max_depth is preferred. Since, with the SP dataset we use a time window of more than "10" days to look back and calculate some of the signals, the max_depth is logically a little higher than the TLS dataset.

Figs 6(b) & 7(b) answer the question of what information gain criteria to choose. And, while the "gini" and "entropy" methods do not impact the training set, "entropy" shows better performance against the validation set.

Figs 6(c) & 7(c) reflects the pruning discussion. The decision tree suffers from the complexity issue and this validation curve highlighted the impact of using higher pruning weight. Both plots follow the same trend of increasing the min_samples_leaf value, the performance of the training set degrades while the validation set performance increases. This is a healthy sign of the ability of the model to generalize, seeing that the model at point "1" scored perfectly for the training set but not very well with the validation set, while it performs very well with point, "15" in the SP curves and "20" in the TLS curves, on both the validation and training sets. The reason for the different optimal points can be explained by the nature of the domains in which the SP and TLS datasets belong. The SP dataset is from a domain where having more data means having a longer history, but not necessarily better data from which to learn. On the other hand, the TLS dataset reflects a domain where having more matches improves the player's knowledge, leading to better accuracy.

C.SVM: Analysis



Fig. 8: SVM LC (a) SP-JPM (b) TLS

1) **Learning Curves:** Fig 8 (a) & (b) show the performance of SVM when adding further data in both datasets. Takeaway points are:

- **Score Fluctuations:** SP shows variable training scores; TLS may show steadier scores.
- **Convergence of Scores:** SP reveals a gap between training and validation scores; TLS may exhibit tighter convergence.
- **Score Levels:** SP F1 scores are moderate; TLS scores need comparison for relative performance.
- **Training Set Size:** No significant validation score improvement in SP beyond 2000 samples; TLS's response to increased data is unknown.
- **Generalization:** SP suggests potential overfitting; TLS may indicate better or worse generalization.
- **Model Suitability:** SP hints at suboptimal complexity; TLS could imply a better fit for the model.

2) **Validation Curves:** For the SVM model, we use the parameters set in table II with their ranges, and the following curves were generated.

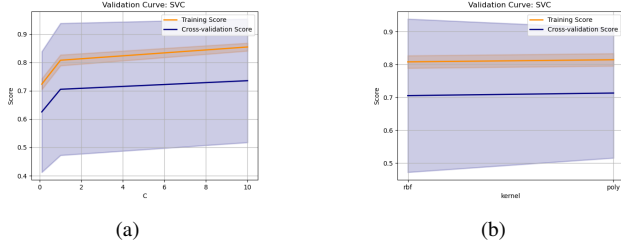


Fig. 9: SP-JPM/SVM (a) C VC (b) Kernel VC

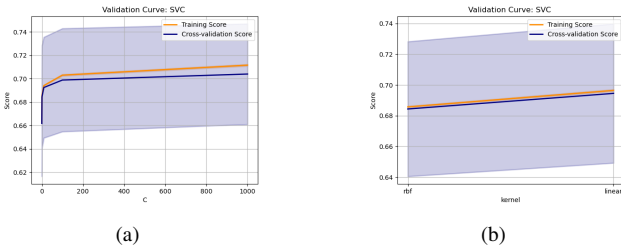


Fig. 10: TLS/SVM (a) C VC (b) Kernel VC

- Validation Curve with 'C' Hyperparameter 9(a) & 10(a): Increasing 'C' enhances training accuracy but increases overfitting risk. Cross-validation score plateaus, indicating marginal gains from a higher 'C'. Consistent performance across data subsets is shown by low score variance.
- Validation Curve with 'kernel' Hyperparameter 9(b) & 10(b): 'rbf' and 'poly' kernels yield comparable cross-validation scores, reflecting similar effectiveness. High training scores with both kernels imply a good fit to the data. The choice of kernel has minimal impact on generalization for this dataset.

D.NN: Analysis



Fig. 11: NN LC (a) SP-JPM (b) TLS

1) Learning Curves:

- Training Score Stability: TLS dataset has more fluctuating training scores; SP dataset likely shows steadier learning.
- Validation Score Improvement: TLS dataset validation scores slowly increase with volatility; SP dataset expected to have a smoother rise in validation scores.

2) **Validation Curves:** For the NN model, we use the parameters set in table II with their ranges, and the following curves were generated.

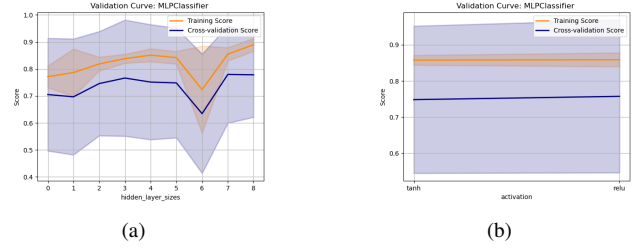


Fig. 12: SP-JPM/NN (a) Hidden Layers Size VC (b) Activation Function VC

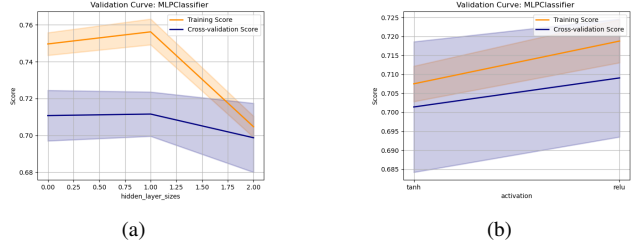


Fig. 13: TLS/NN (a) Hidden Layers Size VC (b) Activation Function VC

- Hidden layer size figs 12(a) & 13(a) shows the impact of changing the neural network complexity. SP plots show a sign of overfitting upon increasing the complexity of the model since we are dealing with a fairly simple problem. TLS plots show better convergence with higher node complexity that can interpolate the different features and their impact on the target.
- Activation function figs 12(b) & 13(b) shows little improvement in the model accuracy for both datasets.

E.kNN: Analysis

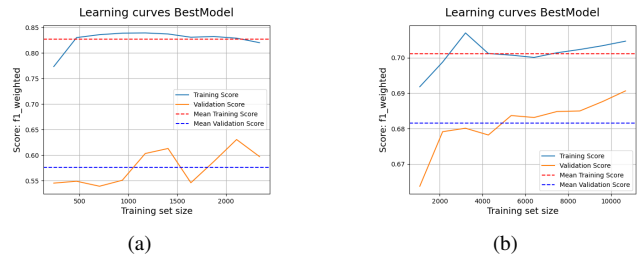


Fig. 14: kNN LC (a) SP-JPM (b) TLS

1) Learning Curves:

- Training Score: SP: The model reaches high training accuracy quickly and maintains it as the training set size increases, indicating good fit to the training data. TLS: Training accuracy improves more gradually and appears less stable, suggesting the model is learning incrementally and may benefit from additional training data.

- **Validation Score: SP:** Validation score shows volatility but an overall upward trend, indicating improving generalization with more data. However, fluctuations might imply the model's sensitivity to specific training subsets. **TLS:** The validation score increases more steadily with fewer fluctuations, which could mean the model is generalizing better as it is exposed to more data.
- **Overall Learning: SP:** The model's learning appears to be more constrained by the dataset's complexity or noise, as indicated by the higher variance in validation scores. **TLS:** The more consistent improvement in validation scores suggests a better match between model complexity and dataset characteristics.
- **Dataset Suitability: SP:** The kNN model may be less suited to the SP dataset or require tuning to handle the data's volatility better. **TLS:** The kNN model seems more suited to the TLS dataset, as evidenced by the more stable learning and generalization curves.

2) **Validation Curves:** For the kNN model, we use the parameters set in table II with their ranges, and the following curves were generated.

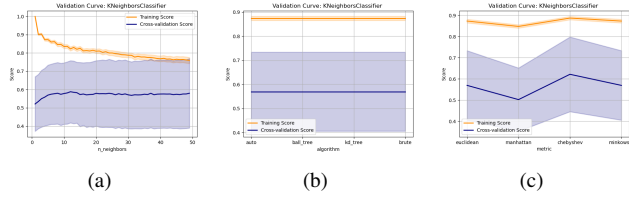


Fig. 15: SP-JPM/kNN (a) N Neighbors VC (b) Algorithm VC (c) Metric VC

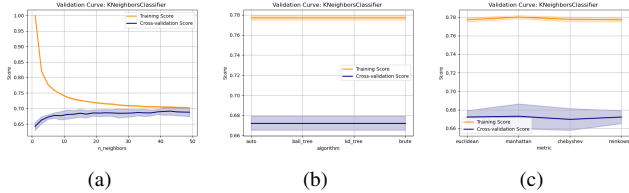


Fig. 16: TLS/kNN (a) N Neighbors VC (b) Algorithm VC (c) Metric VC

- **N Neighbors VC** in figs 15(a) & 16(a) show the effect of changing the number of neighbors on the model. SP plateaus at around 30 while TLS continues to improve without falling into overfitting.
- **Algorithm VC** in figs 15(b) & 16(b) show little to no effect on choosing different algorithms as the criteria for calculating the distance.
- **Metric VC** in figs 15(c) & 16(c) show how the distance metric choice affect the performance. For SP, the choice of error metric seems to have an eradicated performance that translates into the validation score fluctuation. For TLS, the choice has little to no impact on kNN performance.

E. AdaBoost: Analysis

Applying grid search, we found the best model using the tuning parameters mentioned in II.

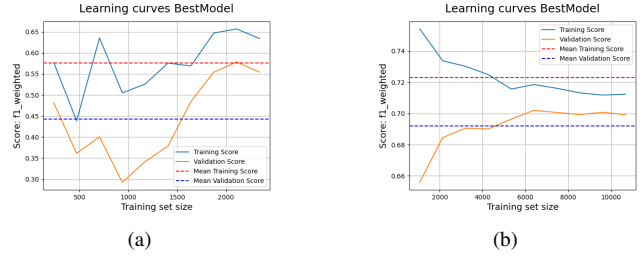


Fig. 17: AdaBoost LC (a) SP-JPM (b) TLS

1) Learning Curves:

- **Training Score: SP:** Exhibits volatile training scores, suggesting inconsistent learning from training data. **TLS:** Shows increasing training scores, indicating consistent learning as the training set size increases.
- **Validation Score: SP:** The validation score is quite volatile, with significant fluctuations indicating potential overfitting or sensitivity to the dataset's complexity. **TLS:** Displays a gradual increase in validation scores, suggesting better generalization with additional data.
- **Overall Learning: SP:** The learning curve suggests that the AdaBoost model has difficulty achieving stable performance, which could be due to the complex nature of the SP dataset. **TLS:** The AdaBoost model appears to learn and generalize more effectively on the TLS dataset, with performance improving as more data is provided.
- **Dataset Suitability: SP:** The dataset's complexity and noise might require a more robust or differently-tuned AdaBoost model. **TLS:** The AdaBoost model seems more suited to the TLS dataset, as indicated by the learning curve's stability and trend.

2) **Validation Curves:** For the AdaBoost model, we use the parameters set in table II with their ranges, and the following curves were generated.

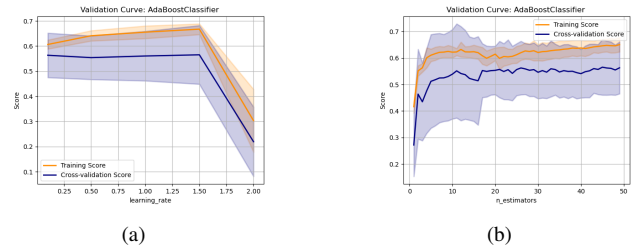


Fig. 18: SP-JPM/AdaBoost (a) Learning Rate VC (b) N Estimators VC

- **Learning rate** figs in 18(a) & 19(a) represent the effect of learning rate on the boosting ability to converge. A higher learning rate opens the model to the vulnerability of missing the global optima. This is the case we see already when we examine the graphs as they both decline rapidly with increasing the learning rate.
- **Number of estimator** figs in 18(b) & 19(b) highlight the boosting mechanism in utilizing several weak learners to contract better overall model. The model performance increases with a higher number of estimators for both datasets with a slight plateauing in the validation score for the TLS dataset at around 20s. The TLS validation curve suggests

Algorithm	Dataset	Training			Validation		Testing		
		Time (sec)	Accuracy	F1	Accuracy	F1	Time (sec)	Accuracy	F1
DT	SP-JPM	0.059281	0.9579	0.9578	0.615	0.6031	0.001114	0.7131	0.7195
	TLS	0.1264	0.7694	0.7693	0.6779	0.6783	0.00261	0.6937	0.6935
SVM	SP-JPM	0.201698	0.7233	0.7239	0.6498	0.6255	0.105348	0.658	0.642
	TLS	5.093985	0.7119	0.7114	0.7049	0.7043	2.337875	0.7086	0.7081
kNN	SP-JPM	0.002638	0.8189	0.8185	0.6064	0.5881	0.032526	0.5997	0.6074
	TLS	0.015089	0.705	0.7048	0.6861	0.6858	0.348207	0.687	0.6869
NN	SP-JPM	2.046987	0.8926	0.8932	0.7931	0.7751	0.001513	0.6402	0.6461
	TLS	51.92254	0.7576	0.7576	0.6966	0.6922	0.007198	0.7084	0.7083
AdaBoost	SP-JPM	0.220162	0.6365	0.6328	0.6062	0.587	0.00586	0.6548	0.6244
	TLS	0.445501	0.7134	0.7132	0.6966	0.6964	0.020268	0.7074	0.7073

TABLE IV: Final Results Summary

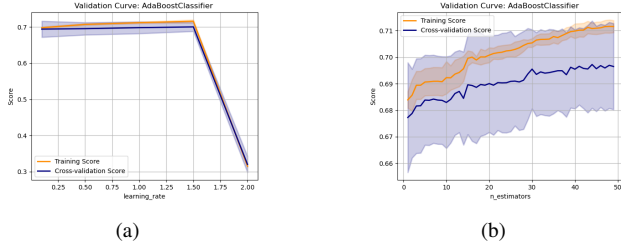


Fig. 19: TLS/AdaBoost (a) Learning Rate VC (b) N Estimators VC

that the model may suffer from overfitting with a higher number of estimators.

V. Conclusion

The table V presents the final parameters selected with respect to datasets and models. These parameters are used to calculate the models' performance table IV.

Algorithm	Hyper Parameters		
	Parameter Name	Best Parameters	
		SP	TLS
Decision Tree (DT)	Max Depth	48	13
	Criterion	entropy	entropy
	Min Samples Leaf	3	15
	C	0.1	1000
Support Vector Machine (SVM)	Kernel	'rbf'	rbf
	N Neighbors	12	43
k-Nearest Neighbor (kNN)	Algorithm	auto	auto
	Metric	minkowski	minkowski
	Alpha	0.0002	0.001
	Learning Rate	constant	constant
Neural Networks (NN)	Hidden Layer Sizes	(10, 10, 10)	(50, 50, 50)
	Activation	tanh	tanh
	N Estimators	34	44
Boosting (AdaBoost)	Learning Rate	0.5	1.5

TABLE V: Final Parameters Tuning

We started the paper by stating that "tls is ideal for assessing the models' performance in a controlled environment with a clear structure and low noise, while the stocks dataset presents a challenge due to its inherent noise and non-stationarity, testing the models' ability to decipher more subtle patterns." in the introduction section. And indeed, our analysis confirmed the inherited noise in the stocks datasets making all the models performance better in general for the validation and testing scores compared to the tennis dataset results. Accordingly, based on the results presented in table IV, we can infer the following:

- **Model Performance:** Neural Networks and Support Vector Machines show superior performance in both training and

testing, indicating their effectiveness in capturing complex patterns.

- **Dataset Complexity:** The lower accuracy and F1 scores on the TLS dataset suggest it has a higher complexity or is more challenging for the models compared to the SP_JPM dataset.
- **Computation Time:** Decision Trees offer the best computation time efficiency during training, while Neural Networks require longer training and testing times, especially on the SP_JPM dataset.
- **Overfitting:** The discrepancy between training and validation accuracy in some models may point towards overfitting, particularly in models with higher complexity.
- **Testing Performance:** The testing performance varies across models, with k-Nearest Neighbors and AdaBoost showing a balance between accuracy and inference speed.
- **Algorithm Suitability:** The varied performance across different datasets indicates that algorithm suitability is highly data-dependent, and model choice should be made accordingly.
- **Occam's Razor:** Given the trade-off between model complexity and performance, simpler models such as Decision Trees may be preferable when they provide adequate performance, in line with the principle of Occam's Razor.

VI. References

- Sipko, Michal, and William Knottenbelt. "Machine learning for the prediction of professional tennis matches." MEng computing-final year project, Imperial College London 2 (2015).
- Pahwa, N., et al. "Stock prediction using machine learning a review paper." International Journal of Computer Applications 163.5 (2017): 36-43.
- Z. K. Lawal, H. Yassin and R. Y. Zakari, "Stock Market Prediction using Supervised Machine Learning Techniques: An Overview," 2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE), Gold Coast, Australia, 2020, pp. 1-6, doi: 10.1109/CSDE50874.2020.9411609.